PROGRAMMING PARADIGMS

DESTRUCTURING, FUNCTIONAL PROGRAMMING, & OOP

AGENDA OVERVIEW

Destructuring

Clean patterns for unpacking data structures into isolated variables seamlessly.

Functional

Declarative computation focusing on pure functions and absolute immutability.

OOP

Modeling system logic around encapsulated state, objects, and structural patterns.

PILLAR ONE

Destructuring Patterns and Unpacking Data

ARRAY AND OBJECT UNPACKING

Object Extraction

Matching keys instantly from complex configurations or JSON payloads without repetitive inline lookups.

```
const { id, settings } = response;
```

Array Extraction

Accessing positional elements cleanly using physical array ordering. Ideal for structured data lists and tuples.

```
const [latitude, longitude] = coordinates;
```

KEY MECHANICS

- Assign default values to protect your applications from undefined returns.
- Rename incoming keys on the fly to avoid scope collision issues.
- Gather remaining parameters cleanly with the modern rest syntax.
- Extract deep properties instantly through nested destructuring layouts.

PILLAR TWO

Functional Programming Concepts and Purity

PURE FUNCTIONS & STATE

Function Purity

Writing deterministic algorithms that return predictable outputs based entirely on inputs without side effects.

```
const multiply = (x, y) => x * y;
```

Immutability

Treating application data as read-only. We construct new objects instead of altering reference memory states.

```
const updated = [...original, newItem];
```

DECLARATIVE OPERATIONS

Filter Method

Maintains collections by retaining values matching strict criteria.

Map Method

Processes elements individually and returns a new populated array.

Reduce Method

Condenses entire sets down into a single comprehensive value.

PILLAR THREE

Object-Oriented Architecture and Design Patterns

THE FOUR PILLARS OF OOP

Encapsulation

Hiding inner complexity, exposing controlled interfaces.

Inheritance

Sharing behavior patterns along parent-child classes.

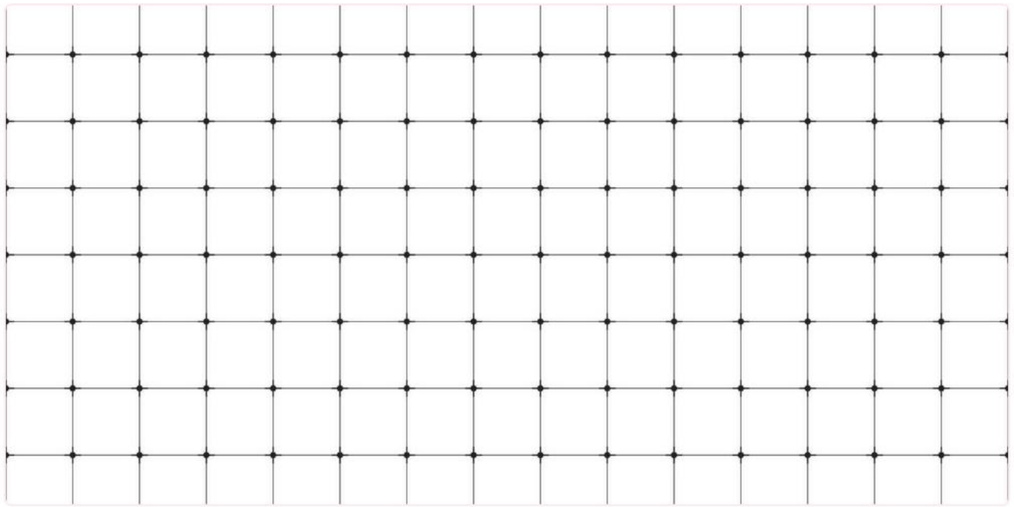
Polymorphism

Dynamic execution of distinct overriding functions.

PARADIGM COMPARISON

Concept	Functional Programming	Object-Oriented Programming
Core Focus	Deterministic transformations, data flows	Encapsulated objects, modeled structures
Data State	Immutable data patterns	Mutable local object states
Flow Control	Declarative maps, filters, reductions	Imperative method calls and algorithms

QUESTIONS?



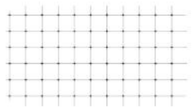
Thank you for your valuable attention.

paradigms.example.com | systems-support@example.com

IMAGE SOURCES

https://png.pngtree.com/thumb_back/fh260/background/20201028/pngtree-abstract-polygonal-line-style-dark-and-red-technology-vector-background-with-image_444192.jpg

Source: pngtree.com



https://static.vecteezy.com/system/resources/previews/067/108/649/non_2x/geometric-grid-pattern-composed-by-squares-and-dot-intersections-abstract-seamless-minimalist-technology-background-vector.jpg

Source: www.vecteezy.com